

Langchain

What is Lang chain?

LangChain is an open source framework that helps in building LLM based applications. It provides modular components and end-to-end tools that help developers build complex AI applications, such as chatbots, question-answering systems, retrieval-augmented generation (RAG), autonomous agents, and more.

1. Supports all the major LLMs
2. Simplifies developing LLM based applications
3. Integrations available for all major tools
4. Open source / Free / Actively developed
5. Supports all major GenAI use cases

LangChain is a software framework that makes it easier to build applications powered by AI language models. Think of it like a toolkit that helps developers quickly put together solutions such as chatbots, customer support assistants, or knowledge search systems.

Instead of writing everything from scratch, LangChain provides ready-made building blocks. This saves time, reduces complexity, and ensures that companies can use AI in practical ways whether it's answering employee questions, helping customers, or automating repetitive tasks.

It's open source, meaning it's free and constantly improved by a global community. And because it works with all the major AI models and tools, businesses don't get locked into one vendor — they can choose what fits best.

Why do we need Langchain?

- **Connecting to External Data** LangChain provides connectors and loaders for APIs, databases, and file systems. This allows LLMs to overcome static training cutoffs by querying **real-time data sources** (e.g., SQL, REST APIs, cloud storage, vector DBs).

Example: Using a Retriever to fetch live financial data from an API and feed it into GPT for analysis.

- **Building Complex Workflows** LangChain's **Chains** and **Agents** enable orchestration of multiple steps. Developers can chain operations like document ingestion → embedding → retrieval → reasoning → tool execution.

Example: A workflow where a user query triggers semantic search, retrieves documents, summarizes them, and then calls an external API for validation.

- **Adding Memory** LangChain supports **ConversationBufferMemory**, **ConversationSummaryMemory**, and **VectorStore-backed memory**. These modules maintain context across turns, enabling stateful interactions.

Example: A chatbot that remembers user preferences across sessions by storing embeddings in Pinecone.

- **Enabling Tool Use** Through **Agents + Tools**, LangChain lets LLMs call external functions, APIs, or utilities. Tools can be calculators, search engines, or custom APIs.

Example: An agent that decides whether to answer directly, run a SQL query, or call a weather API depending on the user's request.

- **Standardizing Development** LangChain enforces consistent abstractions:
 - **LLM wrappers** (uniform interface across models)
 - **Prompt templates** (structured prompt engineering)
 - **Chains** (composable pipelines)
 - **Agents** (decision-making with tool use) This reduces boilerplate and ensures reproducibility across projects.
- **Simplifying RAG Implementation** LangChain integrates with vector databases (FAISS, Pinecone, Weaviate, Milvus) and provides **Document Loaders + Text Splitters + Embedding Models**. This makes Retrieval-Augmented Generation straightforward: ingest docs → embed → store → retrieve → augment LLM context. *Example:* A knowledge assistant that indexes company PDFs into embeddings and retrieves relevant chunks during Q&A.

Benefits of LangChain

1. Concept of Chains

- Compose simple components into powerful workflows
- Sequential execution of LLM calls with intermediate processing
- Built-in chain types for common patterns (e.g., question-answering, summarization)
- Easy debugging and transparency at each step

2. Model Agnostic Development

- Abstract away specific LLM implementations
- Switch between providers (OpenAI, Anthropic, local models) with minimal code changes
- Future-proof applications against model changes
- Consistent interface across different model capabilities

3. Complete Ecosystem

- Document loaders for diverse data sources

- Vector stores for efficient retrieval
- Evaluation tools for measuring performance
- Integration with popular tools (Pinecone, Chroma, Weaviate)
- Growing community extensions and plugins

4. **Memory and State Handling**

- Maintain conversation history automatically
- Multiple memory types (buffer, summary, entity)
- Persist context across sessions
- Control token usage with strategic memory management
- Implement stateful agents for complex interactions

LangChain Application Types

1. **Conversational Chatbots**

- Context-aware chat interfaces with memory
- Domain-specific assistants with custom knowledge bases
- Multi-turn conversation handling
- Personality customization through prompt engineering

2. **AI Knowledge Assistants**

- Document Q&A systems using RAG
- Enterprise knowledge base interfaces
- Self-updating information systems
- Multi-source knowledge synthesis

3. **AI Agents**

- Autonomous systems that reason and plan
- Tool-using agents that can interact with APIs
- Multi-agent systems for complex tasks
- Agents with observation-planning-action loops

4. **Workflow Automation**

- Document processing pipelines
- Content generation and approval systems
- Email categorization and response drafting
- Data extraction from unstructured sources

- Form completion and validation

5. Summarization/Research Helpers

- Document summarization with customizable outputs
- Research paper analysis tools
- Meeting note generators
- News aggregation and summarization
- Literature review assistants

Core Components

1. **Chains** - Combine multiple components for complex workflows
2. **Agents** - Enable LLMs to make decisions and use tools
3. **Memory** - Store and retrieve conversation history
4. **Prompts** - Template and manage inputs to LLMs
5. **Retrievers** - Connect to external data sources
6. **Document Loaders** - Import data from various formats

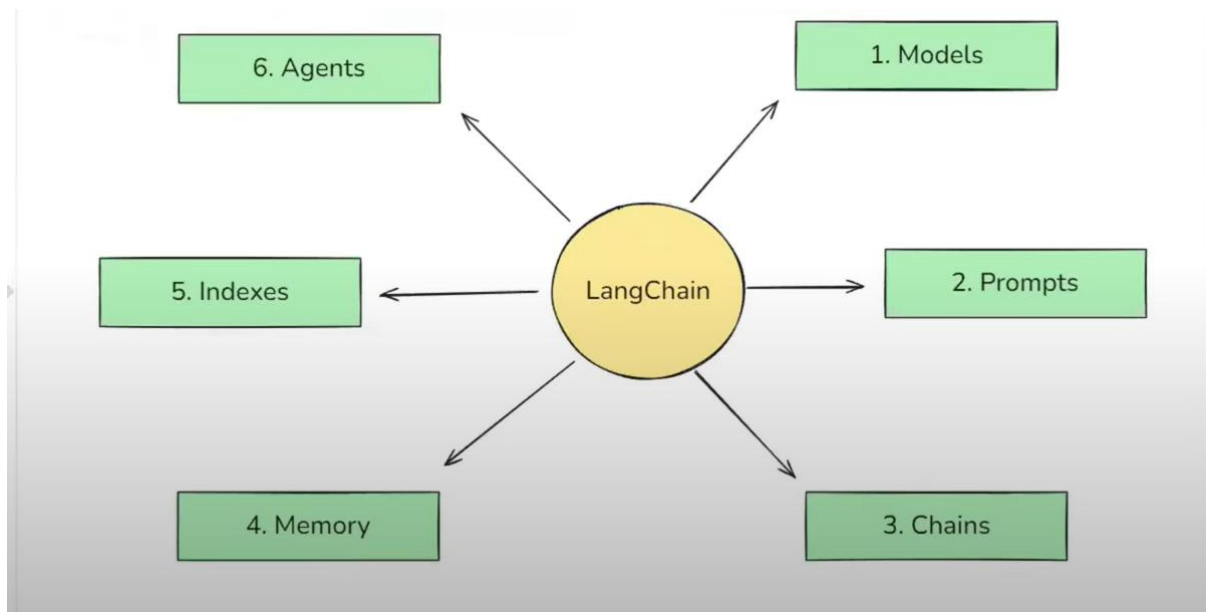
Key Features

- **RAG (Retrieval Augmented Generation)** - Enhance responses with external knowledge
- **Tool Use** - Enable LLMs to interact with external systems
- **Chain-of-Thought** - Break complex reasoning into steps
- **Structured Output** - Parse LLM responses into specific formats
- **Multi-modal Support** - Work with text, images, and other data types

Common Use Cases

- **Chatbots with memory** - Contextual conversation systems
- **Document Q&A** - Answer questions about specific documents
- **Content generation** - Create structured content from prompts
- **Data extraction** - Pull structured data from unstructured text
- **Autonomous agents** - Create AI systems that can plan and execute tasks

Langchain Components



1. Models [LangChain](#)

In LangChain, Models are the core interfaces through which you interact with AI Models.

What are Models in LangChain?

Models in LangChain are standardized wrappers around language models (LLMs) and chat models (like OpenAI's GPT, Cohere, Anthropic, etc.). They provide a consistent interface for interacting with various underlying LLM providers.

Types of Models

- **LLMs (Language Models):** Used for text completion and generation.
- **Chat Models:** Designed for conversational interactions, with messages and roles (e.g., user, assistant).

Purpose

- Abstract away API differences between providers.
- Allow easy swapping of underlying LLMs.
- Enable unified access to text generation, chat, and embeddings.

Example Usage

```
from langchain_openai import ChatOpenAI

# Initialize an OpenAI chat model
llm = ChatOpenAI(model="gpt-3.5-turbo", openai_api_key=OPENAI_API_KEY)

# Generate a response
response = llm.invoke("Explain LangChain models in one sentence.")

print(response.content)
```

Key Benefits

- **Consistency:** Same interface regardless of LLM provider.
- **Flexibility:** Easily change the underlying model without changing your code logic.
- **Integration:** Works seamlessly with other LangChain components (prompts, chains, agents, etc.).

2. Prompts (LangChain)

What are Prompts in LangChain?

Prompts are templates or instructions sent to language models to guide their output. LangChain provides utilities to create, format, and manage prompts, making it easier to build dynamic and complex instructions for LLMs.

Types of Prompts

- **Prompt Templates:** Parameterized templates with placeholders (input variables) that can be filled in at runtime.
- **Few-shot Prompting:** Templates that include example input-output pairs to guide the model with context.

Purpose

- Standardize and reuse prompt structures.
- Dynamically inject user or system data into prompts.
- Enable advanced prompting techniques (few-shot, chain-of-thought, etc.).

Example Usage

1. Dynamic & Reusable Prompts

```
from langchain_core.prompts import PromptTemplate

prompt = PromptTemplate.from_template('Summarize {topic} in {emotion} tone')

print(prompt.format(topic='Cricket', length='fun'))
```

2. Role-Based Prompts

Define the ChatPromptTemplate using from_template

```
chat_prompt = ChatPromptTemplate.from_template([  
    {"system": "Hi you are an experienced {profession}"},  
    {"user": "Tell me about {topic}"},  
])
```

Format the message with variables

```
formatted_messages = chat_prompt.format_messages(profession='Doctor', topic='Viral Fever')
```

3. Few Shot Prompting

Step 1: Examples List

```
examples = [  
    {  
        "input": "I was charged twice for my subscription this month.",  
        "output": "Billing Issue"  
    },  
    {  
        "input": "The app crashes every time I try to log in.",  
        "output": "Technical Problem"  
    },  
    {  
        "input": "Can you explain how to upgrade my plan?",  
        "output": "General Inquiry"  
    },  
    {  
        "input": "I need a refund for a payment I didn't authorize.",  
        "output": "Billing Issue"  
    }  
]
```

Step 2: Create an Example Template

Define the example template with placeholders for input and output

```
example_template = """
```

```
Ticket: {input}
```

```
Category: {output}
```

```
"""
```

Step 3: Build the Few-Shot Prompt Template

Create the few-shot prompt template

```
few_shot_prompt = FewShotPromptTemplate(
```

```
    examples=examples,
```

```
    example_prompt=prompt_template,
```

```
    prefix='Classify the following customer support tickets into categories: Billing Issue, Technical Problem, General Inquiry.',
```

```
    suffix='Ticket: {user_input}\nCategory:',
```

```
    input_variables=["user_input"]
```

```
)
```

Classify the following customer support tickets into one of the categories: 'Billing Issue', 'Technical Problem', or 'General Inquiry'.

Ticket: I was charged twice for my subscription this month.

Category: Billing Issue

Ticket: The app crashes every time I try to log in.

Category: Technical Problem

Ticket: Can you explain how to upgrade my plan?

Category: General Inquiry

Ticket: I need a refund for a payment I didn't authorize.

Category: Billing Issue

Ticket: I am unable to connect to the internet using your service.

Category:

Key Benefits

- **Reusability:** Easily use the same prompt template with different data.
- **Flexibility:** Build complex, multi-step prompts programmatically.
- **Best Practices:** Encourages clean, maintainable prompt engineering.

3. Chains (LangChain)

What are Chains in LangChain?

Chains are sequences of operations that connect prompts, models, and other logic into a workflow. They let you combine multiple steps, such as formatting input, calling an LLM, processing output, and even invoking other tools or APIs.

Types of Chains

- **LLMChain:** The most basic chain—takes input, formats a prompt, sends it to an LLM, and returns the result.
- **SequentialChain:** Runs multiple chains in sequence, passing outputs from one as inputs to the next.
- **SimpleSequentialChain:** Like SequentialChain but only passes the main output between steps.
- **Custom Chains:** You can create your own by composing steps as needed.

Purpose

- Encapsulate multi-step logic.
- Reuse and share workflows.
- Compose complex behaviors from simple building blocks.

Example Usage

```
from langchain.llms import OpenAI

from langchain.prompts import PromptTemplate

from langchain.chains import LLMChain

# Define model and prompt
llm = OpenAI(model="gpt-3.5-turbo")

prompt = PromptTemplate(
    template="Translate '{text}' to French.",
    input_variables=["text"]
)

# Create a chain that puts it together
chain = LLMChain(llm=llm, prompt=prompt)

output = chain.run(text="hello") # Output will be the French translation of "hello"
```

Key Benefits

- **Modularity:** Break complex tasks into manageable steps.
- **Composability:** Combine chains to build advanced applications.

- **Flexibility:** Integrate with memory, tools, and external APIs.

4. Indexes (LangChain)

Indexes connect your application to external knowledge—such as PDFs, websites, or databases.

What are Indexes in LangChain?

Indexes (often called Vector Stores) are components that store and manage document embeddings (numeric representations of text). They enable fast similarity search, retrieval, and lookup of relevant documents or chunks based on a user's query.

Purpose

- Power retrieval-augmented generation (RAG) by letting the LLM access relevant external knowledge.
- Enable semantic search over large collections of text.
- Efficiently store and retrieve documents for context injection.

How It Works

1. **Documents are chunked** (split into pieces).
2. **Embeddings are generated** for each chunk using an embedding model.
3. **Chunks and their embeddings are stored in a vector store** (index).
4. **When a query is made**, it's embedded and compared to the stored vectors to find the most relevant chunks.

Popular Vector Store Backends

- **FAISS** (local, open-source)
- **Pinecone** (cloud, scalable)
- **Chroma** (local, fast)
- **Weaviate, Qdrant, Milvus**, etc.

Example Usage

```
from langchain.vectorstores import FAISS
```

```
from langchain.embeddings import OpenAIEmbeddings
```

```
# Prepare your documents and embeddings
```

```
documents = ["LangChain is powerful.", "Indexes enable retrieval.", "Chains sequence operations."]
```

```
embeddings = OpenAIEmbeddings()
```

```
# Build the vector store (index)
```

```
vectorstore = FAISS.from_texts(documents, embeddings)
```

Perform a similarity search

```
results = vectorstore.similarity_search("How do I retrieve document context?", k=2)
```

Key Benefits

- **Scalability:** Efficiently handles large collections.
- **Semantic Search:** Finds relevant data based on meaning, not just keywords.
- **Integration:** Seamlessly plugged into memory, chains, and agents for RAG workflows.

5. Memory (LangChain)

LLM API calls are stateless

Who is Narendra Modi? → LLM API → Narendra Modi is an Indian politician serving as the 14th and current Prime Minister of India since May 2014

How old is he? → LLM API → As an AI, I don't have access to personal data about individuals unless it has been shared with me in the course of our conversation.

- **ConversationBufferMemory:** Stores a transcript of recent messages. Great for short chats but can grow large quickly.
- **ConversationBufferWindowMemory:** Only keeps the last N interactions to avoid excessive token usage.
- **Summarizer-Based Memory:** Periodically summarizes older chat segments to keep a condensed memory footprint.
- **Custom Memory:** For advanced use cases, you can store specialized state (e.g., the user's preferences or key facts about them) in a custom memory class.

What is Memory in LangChain?

Memory refers to mechanisms for storing and retrieving information across steps in a conversation or workflow. It allows your application to “remember” previous interactions, user inputs, or generated outputs, thereby enabling contextual and coherent multi-turn conversations.

Purpose

- Maintain context across multiple user interactions.
- Enable more dynamic, human-like conversational experiences.
- Store and recall facts, history, or other data for use in chains and agents.

Types of Memory

- **ConversationBufferMemory:** Stores the full conversation as a buffer (like a chat log).
- **ConversationSummaryMemory:** Stores a summary of the conversation, using an LLM to condense previous exchanges.
- **VectorStoreRetrieverMemory:** Uses a vector store (index) to retrieve relevant past context based on similarity.

- **EntityMemory:** Tracks and recalls specific entities mentioned during the conversation.

Example Usage

```
from langchain.memory import ConversationBufferMemory

# Initialize memory
memory = ConversationBufferMemory()

# Save a turn of conversation
memory.save_context({"input": "Who is the CEO of OpenAI?"}, {"output": "Sam Altman"})

# Retrieve memory variables (conversation history)
history = memory.load_memory_variables({})

print(history["history"])
```

Key Benefits

- **Contextual Awareness:** Models can reference previous conversation turns, improving relevance and coherence.
- **Customizability:** Choose the memory type and retention method that fits your application.
- **Seamless Integration:** Easily plug memory into agents, chains, and workflows in LangChain.

6. Agents (LangChain)

What are Agents in LangChain?

Agents are intelligent, autonomous systems that use a language model to decide what actions to take given a user's goal and a set of available tools. Instead of following a fixed sequence, agents reason step-by-step, choosing actions dynamically—like searching the web, running code, or querying a database—to solve complex, multi-step problems.

Purpose

- Automate complex tasks that require decision-making, tool use, or external data fetching.
- Enable conversational assistants that can perform actions, not just generate text.
- Adapt workflows dynamically based on user input or intermediate results.

How Agents Work

1. **Receive a user query or goal.**
2. **Use the LLM to reason about next steps** (e.g., "Should I use a calculator? Search the web? Ask the user for clarification?").
3. **Choose and execute tools** (APIs, search engines, code interpreters, etc.).
4. **Iterate**—the agent may loop, using results from one tool to choose the next action.
5. **Respond to the user** once the task is complete.

Agent Types

- **Zero-shot Agents:** Rely solely on the prompt and LLM for decision-making (no examples given).
- **ReAct Agents:** Use "Reasoning and Acting"—LLM reasons, chooses a tool, observes the result, and repeats.
- **Conversational Agents:** Designed for maintaining dialogue and context.

Tools

Agents use “tools” such as:

- Web search
- Calculators
- Code execution
- Database queries
- Custom APIs

Example Usage

```
from langchain.llms import OpenAI

from langchain.agents import initialize_agent, load_tools

llm = OpenAI(model="gpt-3.5-turbo")
tools = load_tools(["serpapi", "calculator"])
agent = initialize_agent(tools, llm, agent="zero-shot-react-description")

result = agent.run("What is the square root of the population of Canada?")
print(result)
```

Key Benefits

- **Autonomy:** Handles complex, open-ended tasks.
- **Composability:** Combine multiple tools and reasoning steps.
- **Adaptability:** Reacts and plans based on real-time data and results.